

1 Úvod

Dobrý den, dnes Vám budeme představovat náš překladač jazyka IFJ25. Krátce ukážeme architekturu, způsoby implementace a nakonec vybrané technické problémy a jejich řešení.

2 Architektura

Náš překladač má klasickou čtyřfázovou architekturu. Parser si podle potřeby vyžaduje tokeny z lexikální analýzy a z těchto tokenů postupně skládá abstraktní syntaktický strom. Sémantická analýza tento strom zkontroluje – například kontrola definic, použití proměnných, platnost rozsahů a základní typové kontroly nad literály a operátory. Výsledný a ověřený strom pak předáme generátoru kódu, který z něj vytváří odpovídající sadu instrukcí v jazyce IFJcode25.

3 Implementace

Zde můžete vidět vizualizaci jak je řízený náš překladač. O vše se stará hlavní funkce programu, která postupně spouští jednotlivé části. Nejprve je volán parser, který si podle potřeby vyžaduje tokeny z lexéru. Parser postupně vytváří abstraktní syntaktický strom. Jakmile je práce parseru dokončena, je zavolána sémantická analýza, která projde hotový strom a zkontroluje jeho správnost. Jako poslední se spustí generátor kódu, který z AST vytvoří instrukce IFJcode25.

4 Lexér

Lexikální analyzátor generuje tokeny pomocí konečných stavových automatů, které sa delia na primárne a sekundárne. Primárny sa stará o spracovanie jednoduchých tokenov ako **SPECIAL**, **NEWLINE**, **ARITHMETICAL**, **IDENTIFIER** a prázdnych znakov. Zároveň ale deleguje tokeny náročnejšie na spracovanie na sekundárne automaty, ktoré sú určené na špecifické druhy tokenov ako **STRING**, **NUMERICAL** alebo **CMP_OPERATOR**. Využívané sú funkcie uľahčujúce prácu s poľami a makrá na abstrakciu a spriehľadnenie kódu.

5 Syntaktická Analýza

Syntaktická analýza je v našom prekladači riadiaca časť – volá lexikálnu analýzu a z tokenov skladá uzly abstraktného syntaktického stromu. Delíme ju na LL-1 analýzu pre štruktúru programu a precedenčnú analýzu pre výrazy. LL časť je implementovaná ako rekurzívny zostup, kde každá funkcia zodpovedá jednému

neterminálu v našej LL gramatike. Keď LL narazí na začiatok výrazu, odovzdá riadenie precedenčnej analýze.

Precedenčná analýza používa zásobník, na ktorý si ukladá symboly aj vytvorené ASS uzly, a pomocou konvertora si tokeny mapuje na indexy v precedenčnej tabuľke. Podľa toho potom rozhoduje, či urobí posun alebo redukciu. Pri redukcii hľadá značku “<”, spočíta počet symbolov a podľa toho aplikuje pravidlá – buď z jedného symbolu spraví neterminál E, alebo z trojice, ako je (E) alebo E op E, vytvorí nový uzol E. Ak sa k danej kombinácii nedá nájsť pravidlo, analýza skončí chybou.

6 Sémantická analýza

V sémantickú analýzu najprv zaregistrujeme všetky užívateľské funkcie vrátane getterov a setterov a tiež kontrolujeme, či existuje funkcia `main` bez akýchkoľvek parametrov. Potom sa spustí rekurzívny prechod cez ASS, kde vykonávame konkrétne sémantické kontroly. Pri priradení určujeme, či je cieľ lokálny, globálny alebo setter. U volaní funkcií kontrolujeme ich existenciu, správnu aritu a u vestavovaných funkcií tiež typ literálov. U binárnych operácií vykonávame silnú typovú kontrolu hlavne u literálov. Výsledkom je AST doplnený o informácie potrebné pre generovanie.

7 Tabuľka symbolov

Pri sémantickú analýzu potrebuje na kontrolu premenných a parametrov funkcií špeciálny dátový typ. K tomu slúži tabuľka symbolov. Tá je u nás tvorená zásobníkom rozsahov, pričom každý rozsah má svoj AVL strom symbolov. Do neho ukládame názvy premenných. Ak hľadáme nejaký symbol, prechádzame tento zásobník od aktuálneho rozsahu smerom dole, takže tým dodržíme lexikálnu platnosť. Na obrázku je vidieť príklad: v scope 2 sú uložené symboly ako `result`, `str` alebo `i`, ktoré môže sémantika v tomto rozsahu nájsť.

8 Generovanie medzikódu

Stručne: generátor raz prejde AST, vytvorí zoznam inštrukcií IFJcode25 a potom tento zoznam vypíše ako výsledný program.

Generátor predstavuje posledný krok prekladača: vezme spracovaný program (AST) a preloží ho do IFJcode25 – výstup na štandardnom výstupe.

Prechádza celý AST v `generate(tree)`: Generuje skok na požadovanú bezparametrickú funkciu `main` a vytvorí všetky funkcie.

Výstup – ukladaný ako obojsmerný zoznam inštrukcií s trojadresným kódom (`TAClist tac`). Zámerom boli jednoduchšie optimalizácie generovaného kódu

(neimplementované). Nakoniec sa vytlačí prológ a potom inštrukcie.

Pre každú funkciu (`gen_func_def`):

- vytvorí návestie (napr. `$foo` alebo `$$main` pre vstupnú funkciu),
- pripraví rámce a priestor pre návratovú hodnotu
- deklaruje lokálne premenné a parametre
- následne vygeneruje telo a epilóg funkcie

Pri generovaní príkazov (`gen_stmt`) má funkcie, ktoré prekladajú:

- priradenia do premenných,
- `if` a `while` na kombinácie návestí a podmienených skokov,
- `return` na presun výrazu do `LF@retval1` a vykonanie `POPFRAME + RETURN`,
- volania funkcií na vytvorenie dočasného rámca , odovzdanie argumentov a vykonanie `CALL`.

Výrazy (`gen_expr`) generujú:

- literály ako `IFJcode` konštanty,
- identifikátory ako správne pomenovania premenných (globálne/lokálne),
- binárne operátory ako sekvencie inštrukcií, ktoré zároveň vynucujú jazykové pravidlá (kontroly typov, konkatenácia refazcov vs. numerické sčítanie atď.).

Sleduje aj informácie o rozsahoch a lokálnych premenných (`scope_depth`, `locals[...]`, `textttpending_locals[...]`) tak, aby boli mená premenných jednoznačné v rámci úrovne rozsahu a aby sa `DEFVAR` pre lokálne premenné vkladali na správne miesto v blokoch a cykloch.

9 Problémy

9.1 Syntax

Po implementácii syntaktickej analýzy sa naskytla otázka, ako predávať informácie do ASS. Dohodli sme sa že budem uzly naplnat priebezne pocas overovania spravnosti gramatiky. Znamenalo to aj úpravu zásobníka pre potreby ukladania si priebežného stavu ASS počas prekladu výrazu precedenčných analýz.

9.2 Sémantika

Při implementaci sémantické analýzy na definice funkcí se vyskytl problém se správným vytvářením rozsahu platnosti, protože parametry funkce nebyly v rozsahu těla vidět. Funkce totiž vytvořila vlastní rozsah, do kterého vložila své

parametry a následně volala funkci pro analýzu bloku, který taktéž vytvořil vlastní rozsah. Jako řešení jsme implementovali to, že při zpracování funkce nevytváříme pro tělo nový rozsah pomocí bloku, ale tělo funkce projdeme přímo v rámci jednoho rozsahu, do kterého jsme vložili parametry. Díky tomu jsou parametry i lokální proměnné v jednom společném rozsahu platnosti.

Note: The English part of this document was translated with AI (GPT-5.3). The translation has been reviewed, but minor inconsistencies may remain.

1 Introduction

Today, we will present our IFJ25 language compiler. We will briefly present the architecture, the implementation approach, and, finally, selected technical problems and their solutions.

2 Architecture

Our compiler follows a classic four-phase architecture. The parser requests tokens from the lexical analysis as needed and gradually builds an abstract syntax tree from them. The semantic analysis then checks this tree — for example, definitions, variable usage, scope validity, and basic type checks on literals and operators. The verified tree is then passed to the code generator, which produces the corresponding set of instructions in IFJcode25.

3 Implementation

Here you can see a visualisation of how our compiler is controlled. Everything is managed by the main program function, which sequentially runs individual parts. First, the parser is called, which requests tokens from the lexer when needed. The parser gradually constructs the abstract syntax tree. Once parsing is finished, semantic analysis is called, which walks through the finished tree and checks its correctness. Finally, the code generator runs and produces IFJcode25 instructions from the AST.

4 Lexer

The lexical analyser generates tokens using finite state machines divided into primary and secondary ones. The primary automaton handles simple tokens such as `SPECIAL`, `NEWLINE`, `ARITHMETICAL`, `IDENTIFIER`, and whitespace. More complex tokens are delegated to secondary automatons, which process specific token types such as `STRING`, `NUMERICAL`, or `CMP_OPERATOR`. Helper functions for working with arrays and macros are used to simplify the implementation and make the code clearer.

5 Syntactic Analysis

Syntactic analysis is the controlling part of the compiler — it calls lexical analysis and builds nodes of the abstract syntax tree from tokens. We divide it into LL(1) analysis for program structure and precedence analysis for expressions.

The LL part is implemented as recursive descent, where each function corresponds to one nonterminal in our LL grammar. When the LL parser encounters the beginning of an expression, control is passed to the precedence analysis.

The precedence analysis uses a stack to store symbols and create AST nodes, and a converter to map tokens to indices in the precedence table. Based on this, it decides whether to shift or reduce. During reduction, it looks for the “<” marker, counts the number of symbols, and applies rules accordingly — either converting one symbol into the nonterminal E, or creating a new node from three symbols such as (E) or E op E. If no rule matches the combination, the analysis ends with an error.

6 Semantic analysis

In semantic analysis, we first register all user-defined functions, including getters and setters, and also check that a parameterless main function exists. Then, a recursive traversal of the AST is performed, during which specific semantic checks are executed. During the assignment, we determine whether the target is local, global, or a setter. For function calls, we check their existence, correct arity, and for built-in functions, also literal types. Binary operations perform strict type checking, mainly for literals. The result is an AST extended with information needed for code generation.

7 Symbol table

During semantic analysis, a special data structure is required to check variables and function parameters. For this purpose, we use a symbol table implemented as a stack of scopes, where each scope contains its own AVL tree of symbols. Variable names are stored in these trees. When searching for a symbol, the stack is traversed from the current scope downward, which preserves lexical scope rules. In the example, scope 2 contains symbols such as result, str, or i, which semantic analysis can find within that scope.

8 Intermediate code generation

Briefly: the generator walks through the AST once, creates a list of IFJcode25 instructions, and then prints this list as the final program.

The generator is the final step of the compiler: it takes the processed program (AST) and translates it into IFJcode25, which is printed to standard output.

It traverses the entire AST in `generate(tree)`: It generates a jump to the required parameterless function `main` and creates all functions.

Output — stored as a doubly linked list of instructions using three-address code (`TAClist tac`). The intention was to allow simple optimizations of the generated code (not implemented). Finally, the prologue is printed followed by all instructions.

For each function (`gen_func_def`):

- creates a label (e.g. `$foo` or `$$main` for the entry function),
- prepares frames and space for the return value,
- declares local variables and parameters,
- then generates the function body and epilogue

When generating statements (`gen_stmt`), functions translate:

- assignments to variables,
- `if` and `while` into combinations of labels and conditional jumps,
- `return` into moving the expression to `LF@%retval1` followed by `POPFRAME + RETURN`,
- function calls into creating a temporary frame, passing arguments, and executing `CALL`.

Expressions (`gen_expr`) generate:

- literals as IFJcode constants,
- identifiers as correctly named variables (global/local),
- binary operators as instruction sequences that also enforce language rules (type checks, string concatenation vs numeric addition, etc.).

It also tracks scope and local variable information (`scope_depth`, `locals[...]`, `pending_locals[...]`) so that variable names remain unique within a scope level and `DEFVAR` for local variables is inserted at the correct place in blocks and loops.

9 Problems

9.1 Syntax

After implementing syntactic analysis, the question arose of how to pass information to the AST. We agreed to fill AST nodes incrementally while verifying grammar correctness. This also required modifying the stack to store the intermediate AST state during expression parsing in precedence analysis.

9.2 Semantics

During the implementation of semantic analysis for function definitions, a problem appeared with correct scope creation, because function parameters were not visible inside the function body. The function created its own scope, inserted parameters into it, and then called block analysis, which created another scope. The solution was to avoid creating a new scope for the function body and instead process the body directly within the same scope that already contains the parameters. This ensures that parameters and local variables share the same scope.